

J2OS Visual Training Document for Users

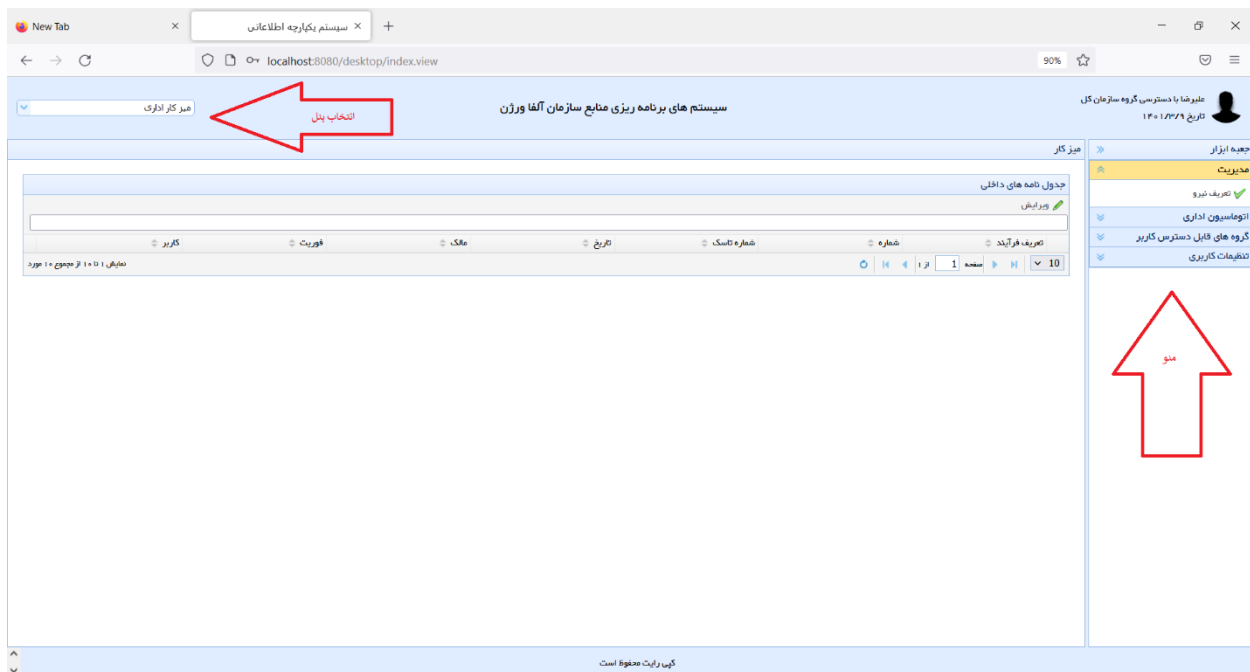
Prepared by: Java School Development and Research Team (J2OS)

Date: February 24, 2025

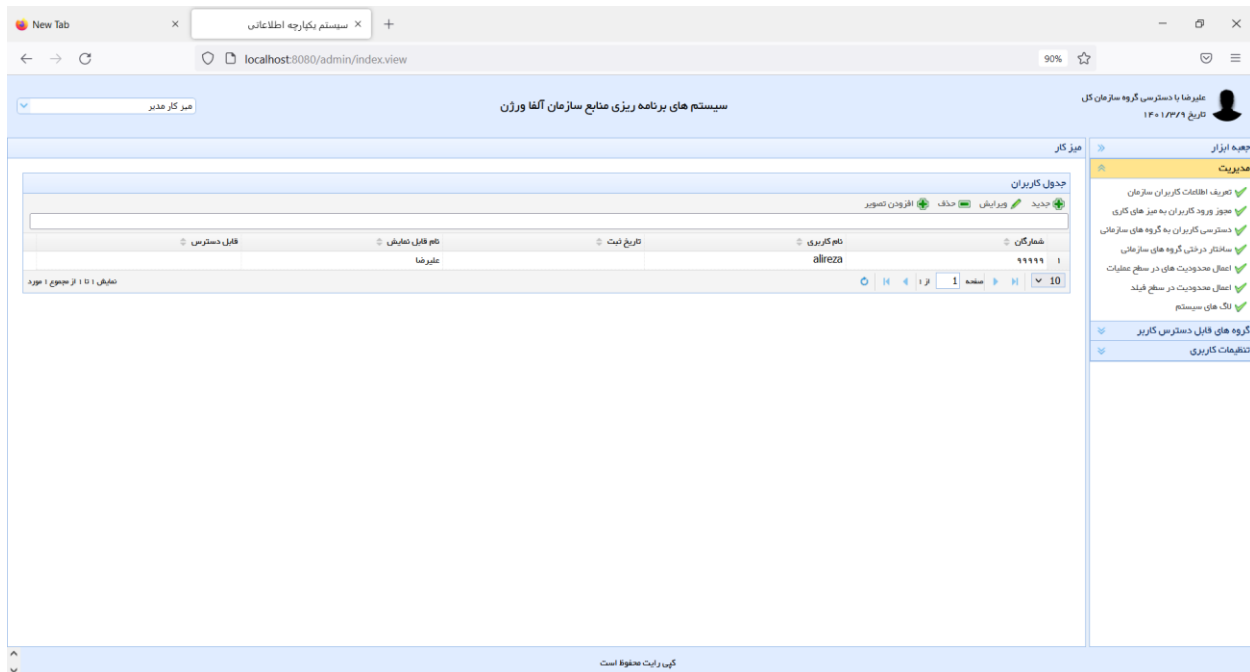
Audience: Enterprise users, managers, and operational teams

Software Usage Training

To begin, log into the software using the admin username and password, which you can obtain from the project coordinator. Note that in this software, you can create as many admins as needed, categorized however you like. In this document, the login address is <http://localhost:8080>. After logging in, you will see a screen similar to the one below.

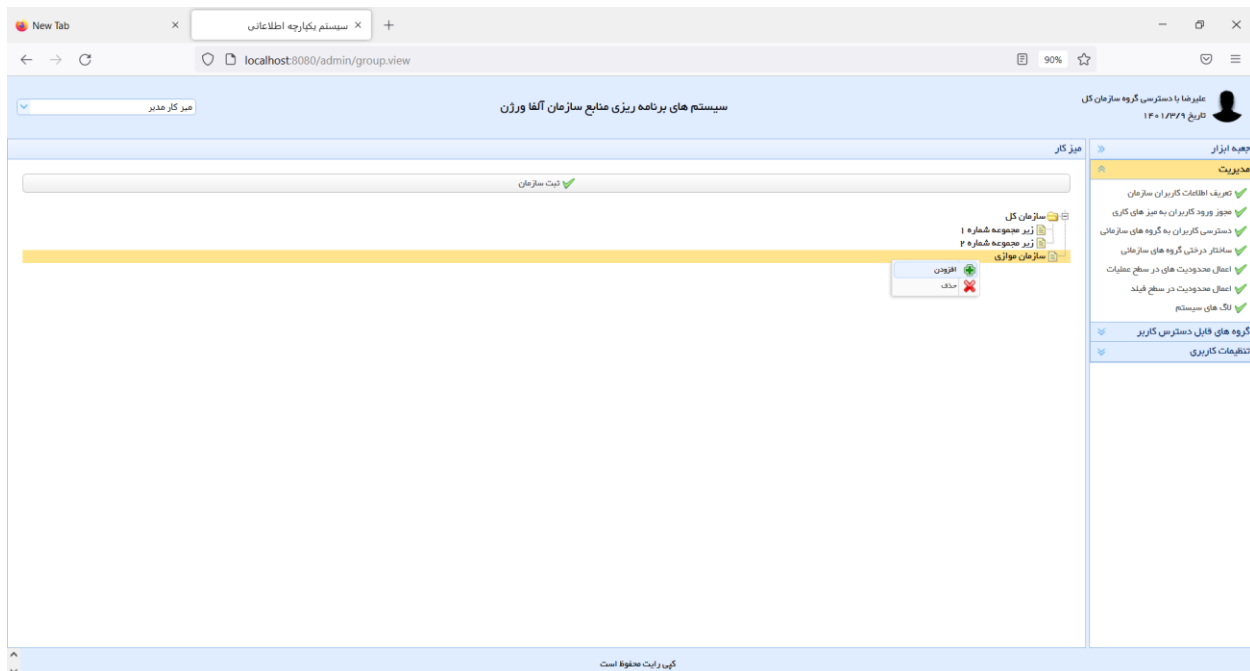


Now, go to the "Panel" section and select "Admin Dashboard."

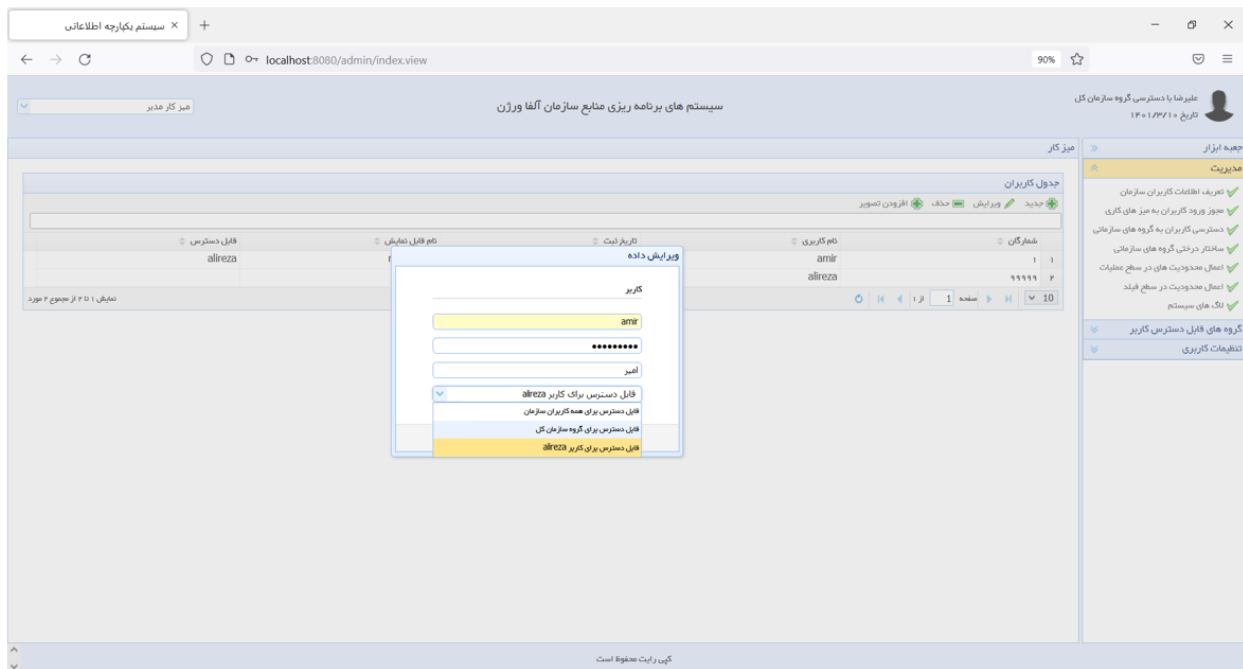


Step 1: Defining the Organizational Structure

As the system admin, your first task is to define the organizational structure. To do this, go to the "Organizational Group Tree Structure" section.



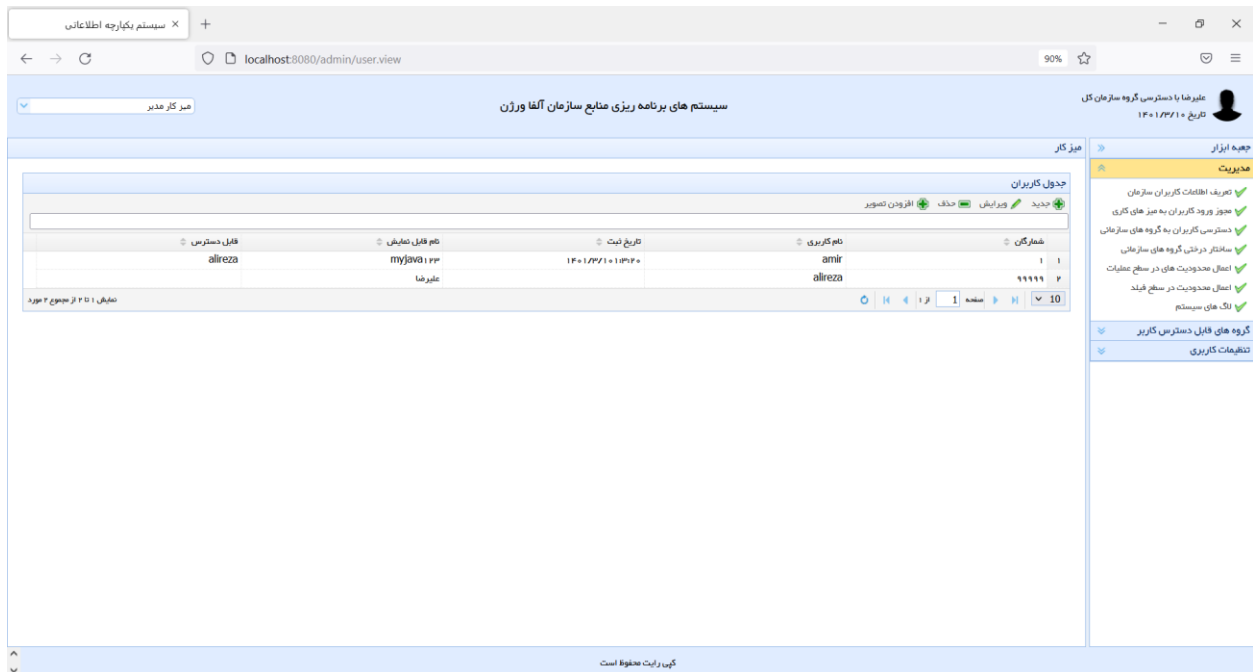
As you can see, to create a new organization, use the "Register Organization" button. To add a subgroup, right-click on the organization and select "Add." In this document, one main organization called "Main Organization" and two subgroups named "Subgroup No. 1" and "Subgroup No. 2" have been created. The examples in this document are based on these two subgroups. Keep in mind that the organizational structure, including individual roles, must be defined—this can happen over time. Now, go to the "Define Organizational Users" section.



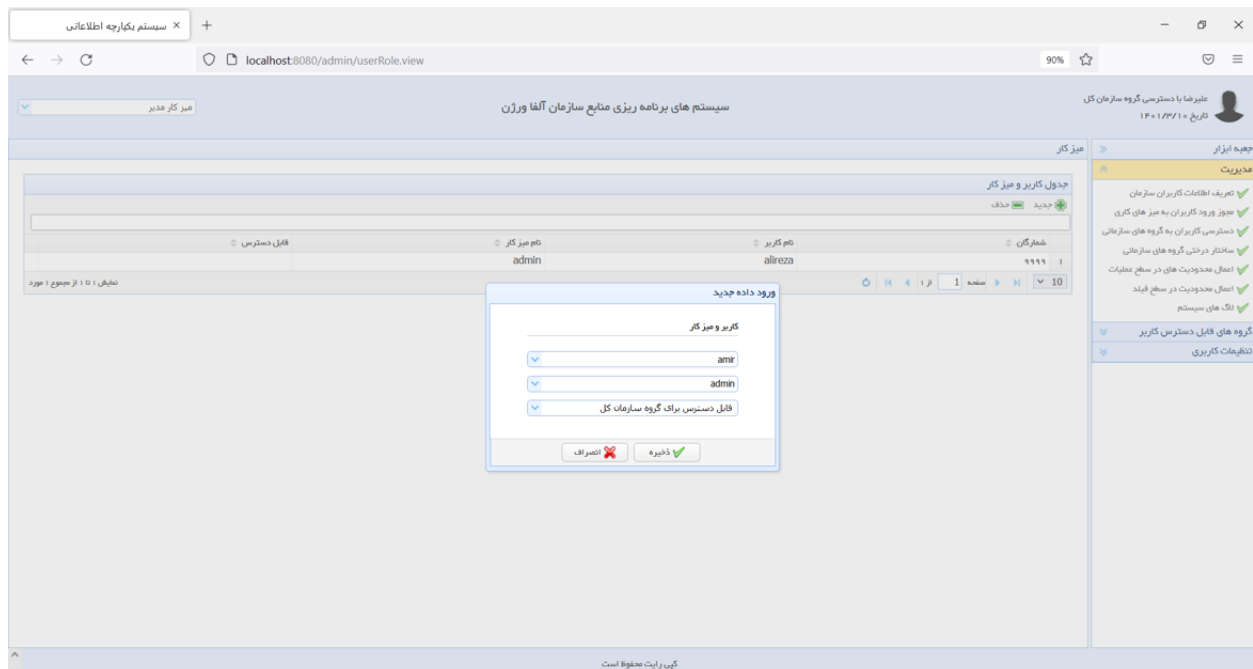
In this example, a user named "Amir" was created. Note that the confidentiality level determines who can see this user within the system:

- **Level 1:** Visible to all organizational users (meaning any user who enters the admin panel can see this user).
- **Level 2:** Visible only to the "Main Organization" group (since Amir is only in this group, he can limit visibility to his own group).
- **Level 3:** Visible only to Amir himself.

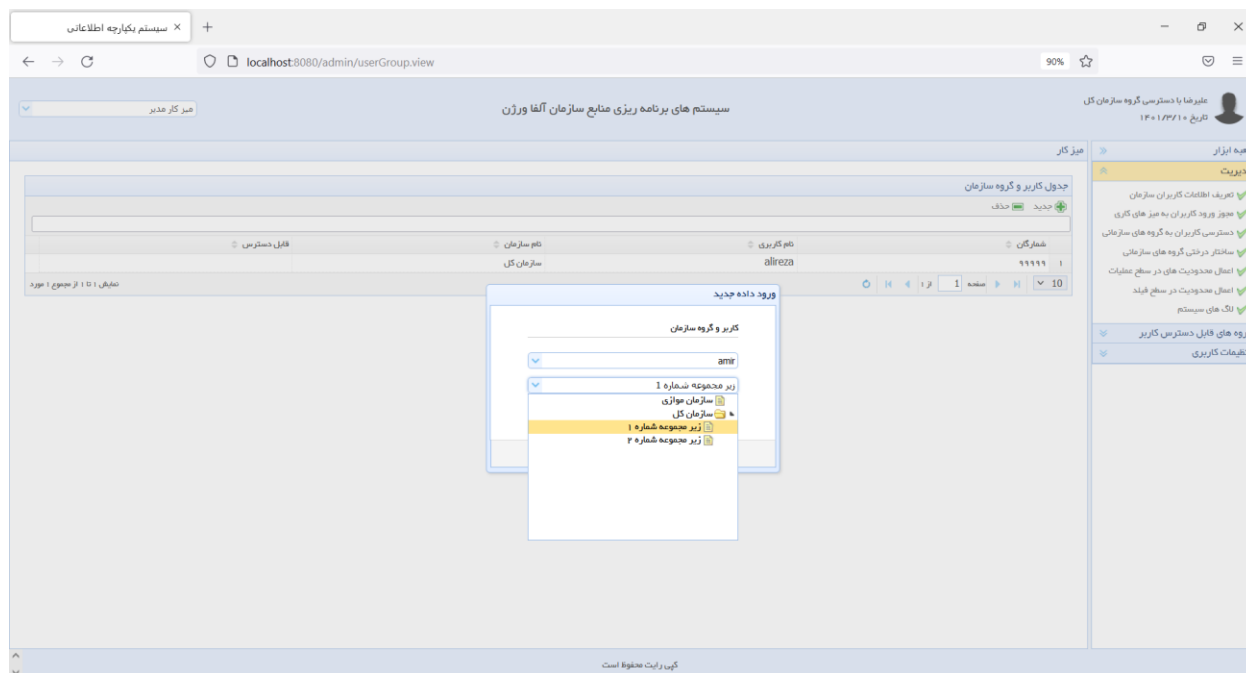
This option is mandatory when entering user data or anywhere else in the system. **Note:** Level 2 visibility depends on the user's group.



The user has been created. Now, go to the "User Dashboard Permissions" section to grant the user access to dashboards.

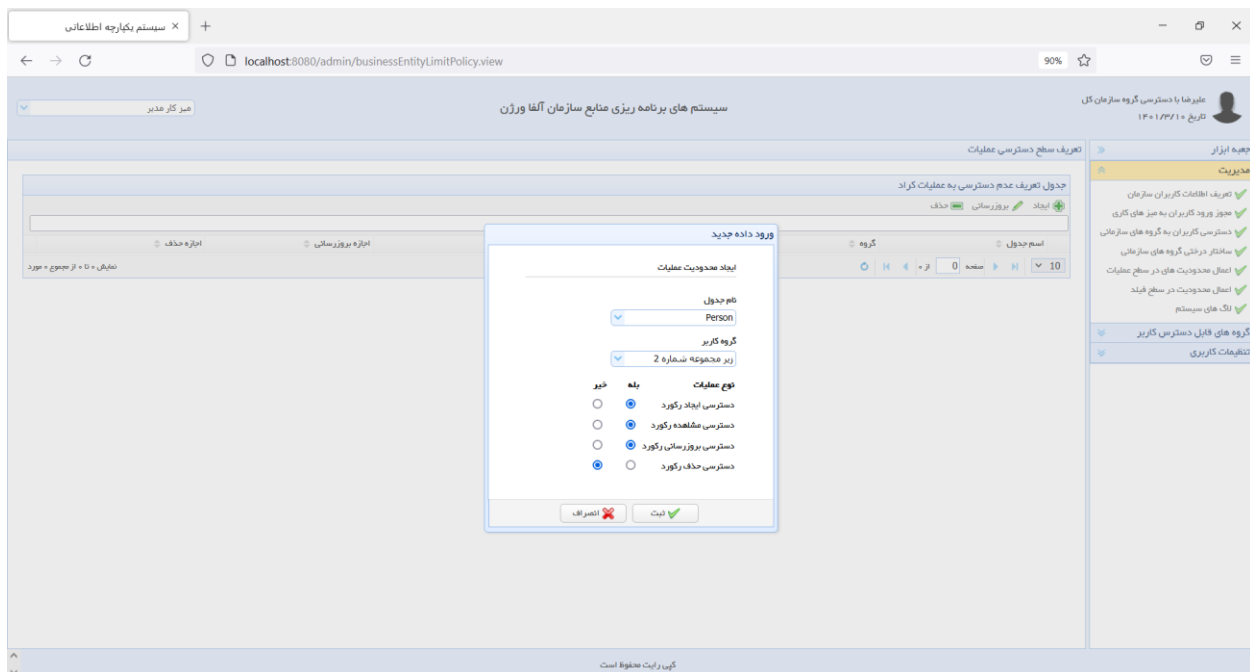


Note that at this stage, we've granted the user access to the "Admin Panel." All users have access to the "Administrative Dashboard" by default. Next, we need to enable the user's access to the organizational tree. To do this, go to the "User Access to Organizational Groups" menu.



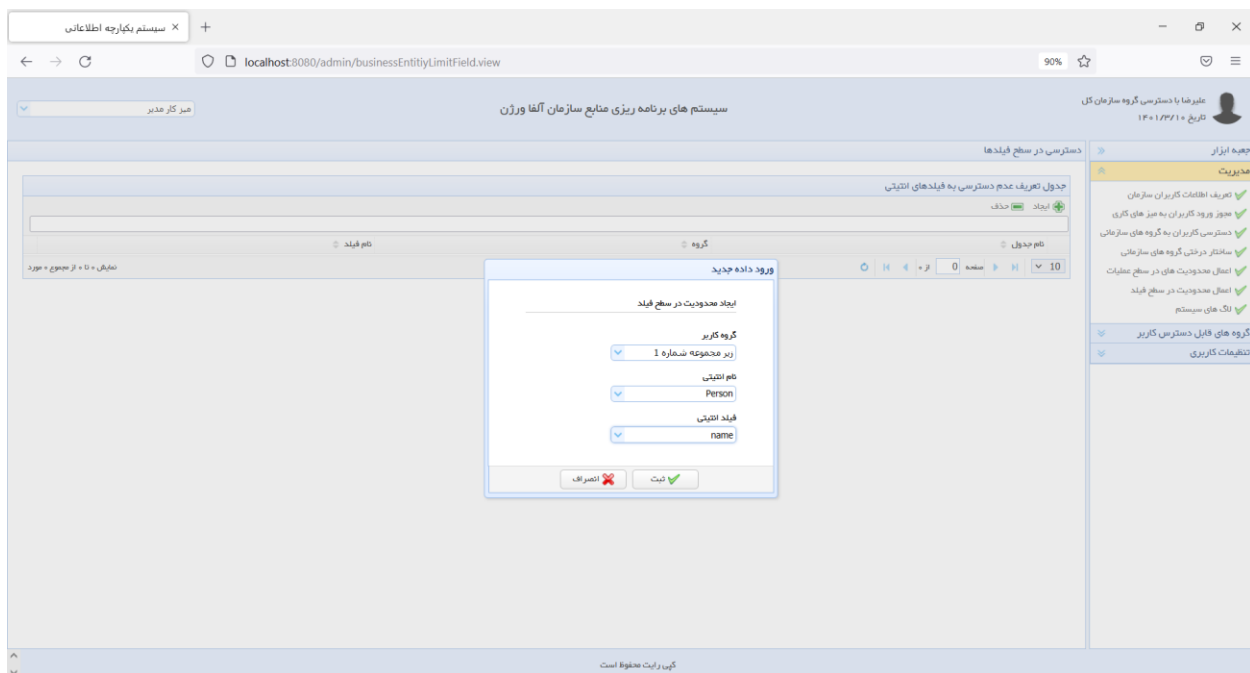
At this stage, the user "Amir" has been granted access to the "Subgroup No. 1" and "Subgroup No. 2" organizational groups.

In the "Apply Restrictions at Operation Level" section, you can limit a user's access to software entities.



In this example, "Subgroup No. 2," where Amir is a member, does not have "Delete" access to records in the "Person" table (or entity). In this section, you only define restrictions.

Now, proceed to the "Apply Restrictions at Field Level" menu. Here, you can specify which organizational group cannot access specific fields of an entity. As with the previous concept, you define the restrictions.



As you can see, "Subgroup No. 1" has no access to the "name" field of the "Person" entity.

سیستم بکارچه اطلاعاتی

localhost:8080/admin/businessEntityLimitPolicy.view

سیستم های برنامه ریزی منابع سازمان آلفا ورژن

مدیر کار مدیر

عریف دسترسی عملیات

جدول تعریف عدم دسترسی به عملیات گراد

ایجاد بروزرسانی حذف

اسم جدول	گروه	اجازه ثبت	اجازه دیدن	اجازه بروزرسانی	اجازه حذف
Person	زیر مجموعه شماره ۲	true	true	true	false

نمایش ۱ تا ۱ از مجموع ۱ مورد

چهره ابزار

مدیریت

- تعریف اطلاعات کاربر از سازمان
- مجوز ورود کاربر از به سز های کاری
- دسترسی کاربر از به گروه های سازمانی
- ساختار درختی گروه های سازمانی
- اعمال محدودیت های در سطح عملیات
- اعمال محدودیت در سطح فرایند
- لگ های سیستم

گروه های قابل دسترسی کاربر

تنظیمات کاربری

گوشه سایت محفوظ است

All operations performed by the admin or other users in the system can be tracked and viewed in the "Logs" section. This makes it very easy to detect any violations within the system.

سیستم بکارچه اطلاعاتی

localhost:8080/admin/businessEntityLog.view

سیستم های برنامه ریزی منابع سازمان آلفا ورژن

مدیر کار مدیر

لگ ها

لگ های سیستم

نمایش

شمارگان	نام کاربر	عملیات	تاریخ عملیات	موجودیت
۱	alireza	INSERT	۱۴۰۱/۰۳/۰۸:۰۵:۳۷	Group
۲	alireza	INSERT	۱۴۰۱/۰۳/۰۸:۰۵:۱۳	Group
۳	alireza	INSERT	۱۴۰۱/۰۳/۰۸:۰۵:۰۴	Group
۴	alireza	INSERT	۱۴۰۱/۰۳/۰۸:۰۴:۴۹	Group
۵	alireza	UPDATE	۱۴۰۱/۰۳/۰۸:۰۴:۴۹	Group
۶	alireza	DELETE	۱۴۰۱/۰۳/۰۸:۰۵:۱۱	Group
۷	alireza	INSERT	۱۴۰۱/۰۳/۰۸:۰۵:۱۶	Group
۸	alireza	UPDATE	۱۴۰۱/۰۳/۰۸:۰۵:۱۶	Group
۹	alireza	DELETE	۱۴۰۱/۰۳/۰۸:۱۱:۱۴	Group
۱۰	alireza	INSERT	۱۴۰۱/۰۳/۰۸:۱۱:۱۴	Group

نمایش ۱ تا ۱۰ از مجموع ۲۲ مورد

چهره ابزار

مدیریت

- تعریف اطلاعات کاربر از سازمان
- مجوز ورود کاربر از به سز های کاری
- دسترسی کاربر از به گروه های سازمانی
- ساختار درختی گروه های سازمانی
- اعمال محدودیت های در سطح عملیات
- اعمال محدودیت در سطح فرایند
- لگ های سیستم

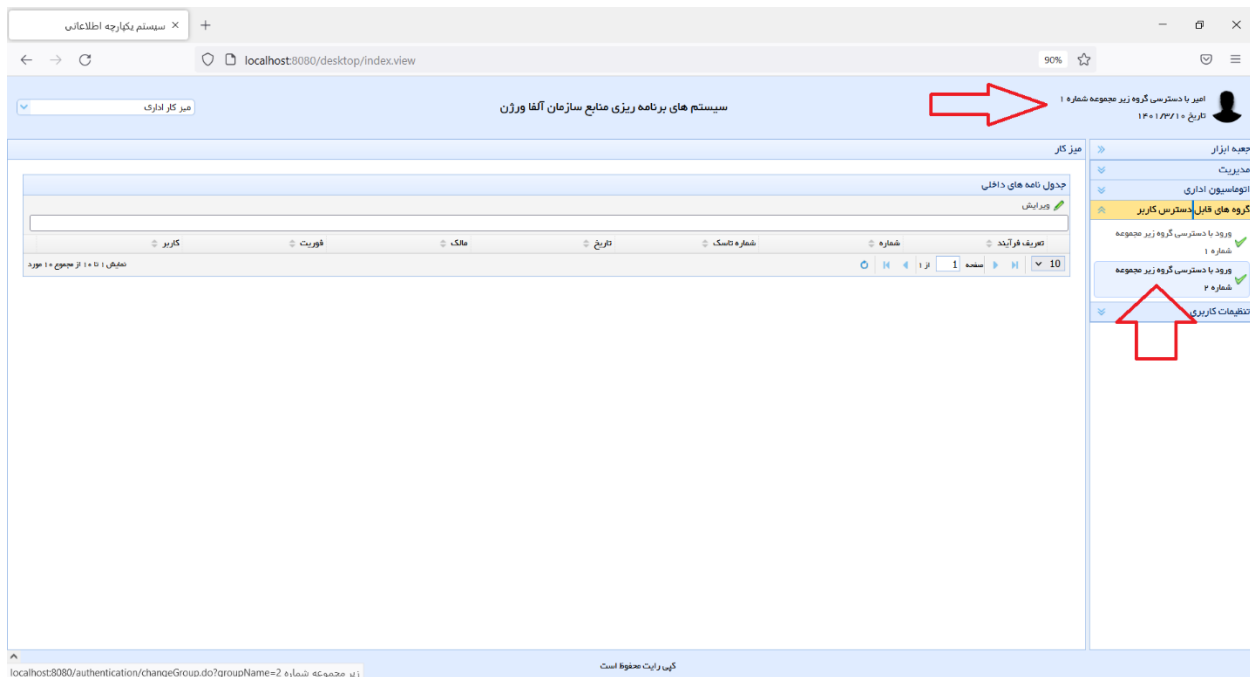
گروه های قابل دسترسی کاربر

تنظیمات کاربری

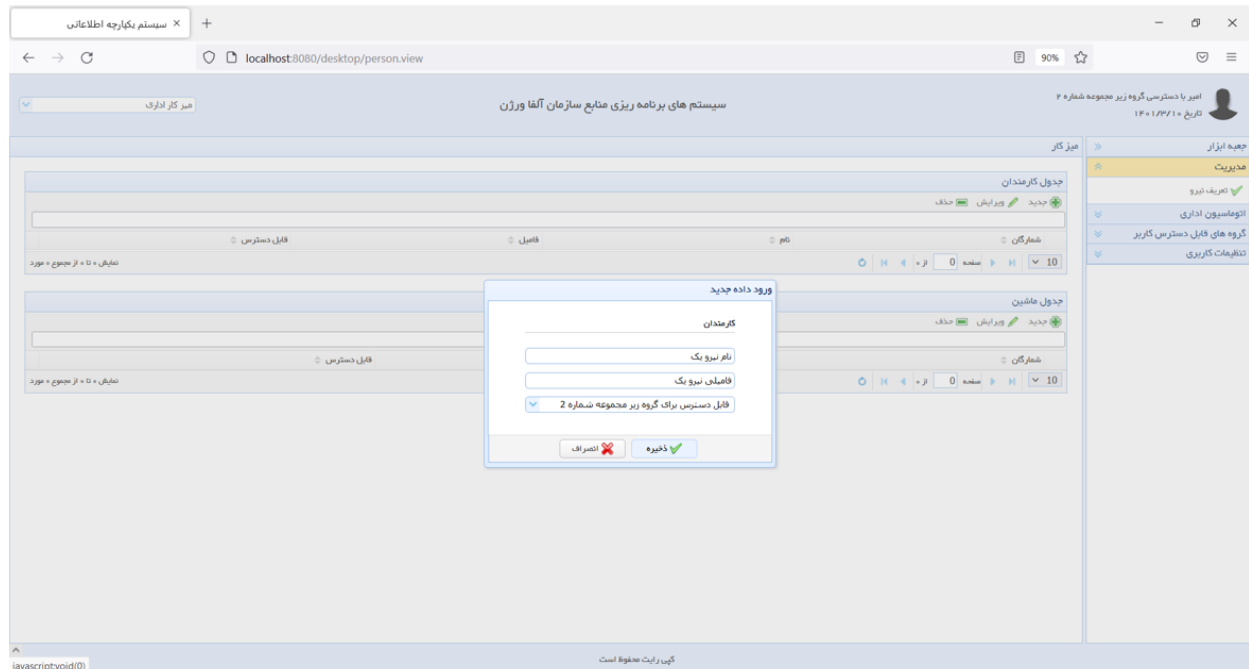
گوشه سایت محفوظ است

The system even logs viewing activities. Now, using the "User Settings" menu, click "Log Out" and log in as the user "Amir." If you recall, Amir has access to the admin panel and is a member of two organizational groups. He can view his organizational roles in the "User Accessible Groups" menu and click on the desired group to review information from that group's perspective.

By clicking on an organizational group, the selected group is displayed at the top of the page. Now, Amir enters the "Admin Panel."

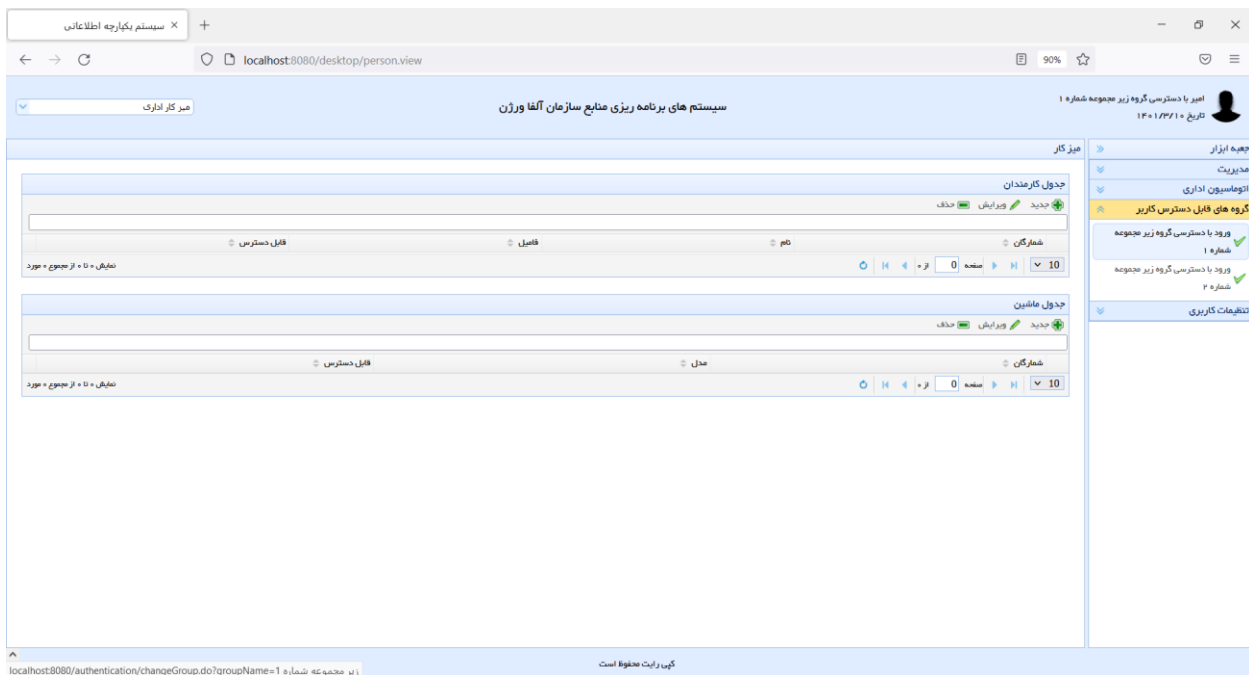


Go to the "Define Personnel" section. This section is a simple CRUD interface designed for training purposes. Here, the "Person" and "Car" tables have a Many-to-Many relationship, facilitated by an intermediary table. First, select "Subgroup No. 2" for the user by clicking on the "User Accessible Groups" menu.



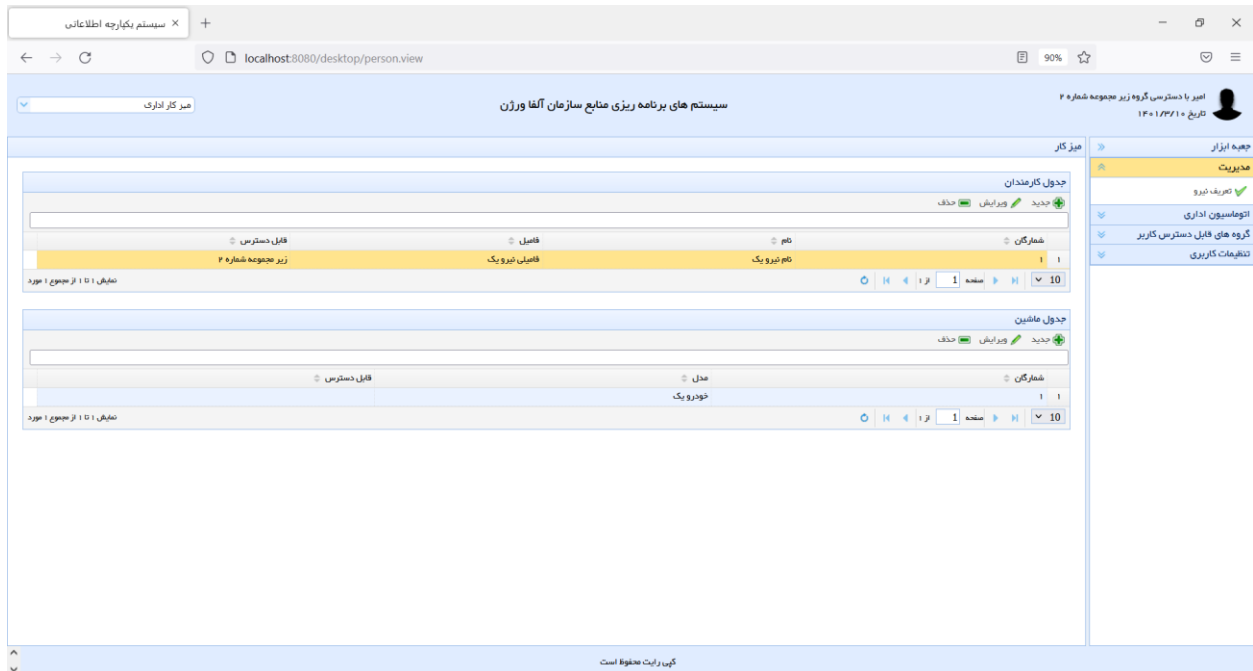
In this section, we define an employee and make this record accessible to "Subgroup No. 2."

If a user with access to "Subgroup No. 1" logs in, they won't be able to see this employee's information.

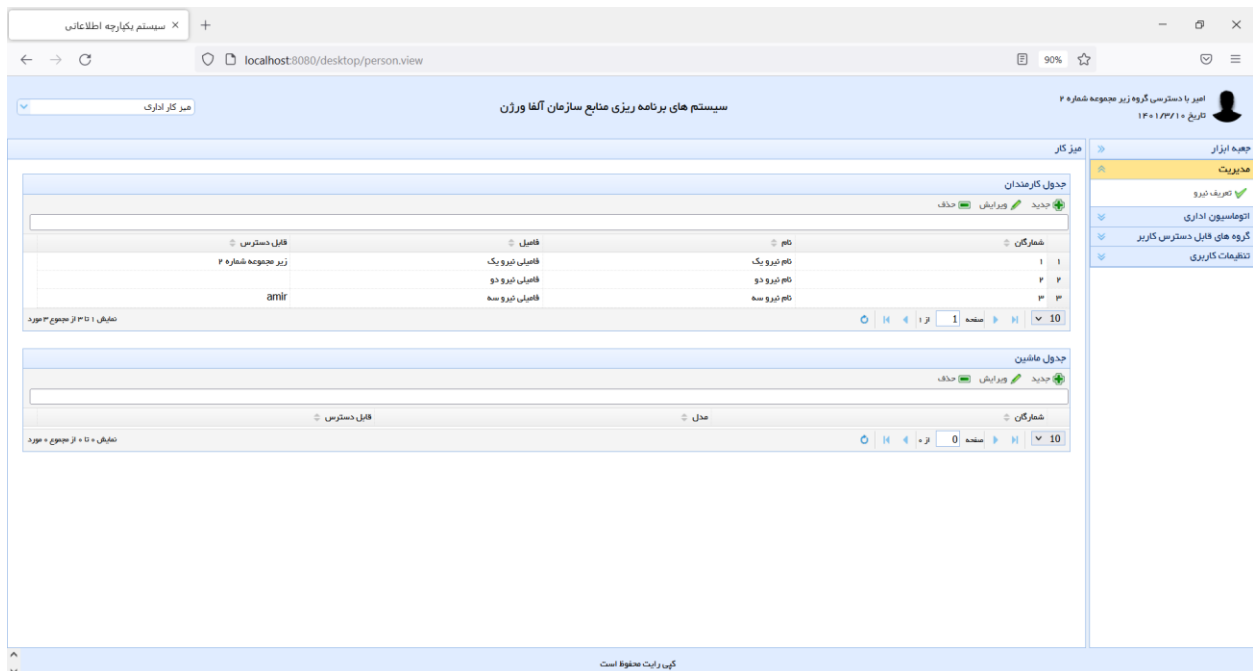


Now, change Amir's organizational group to view the inserted record.

Click on "Employee No. 1," then add a vehicle to this employee.



Now, add two more employees and set their data visibility to "Entire Organization" and "Amir Only."



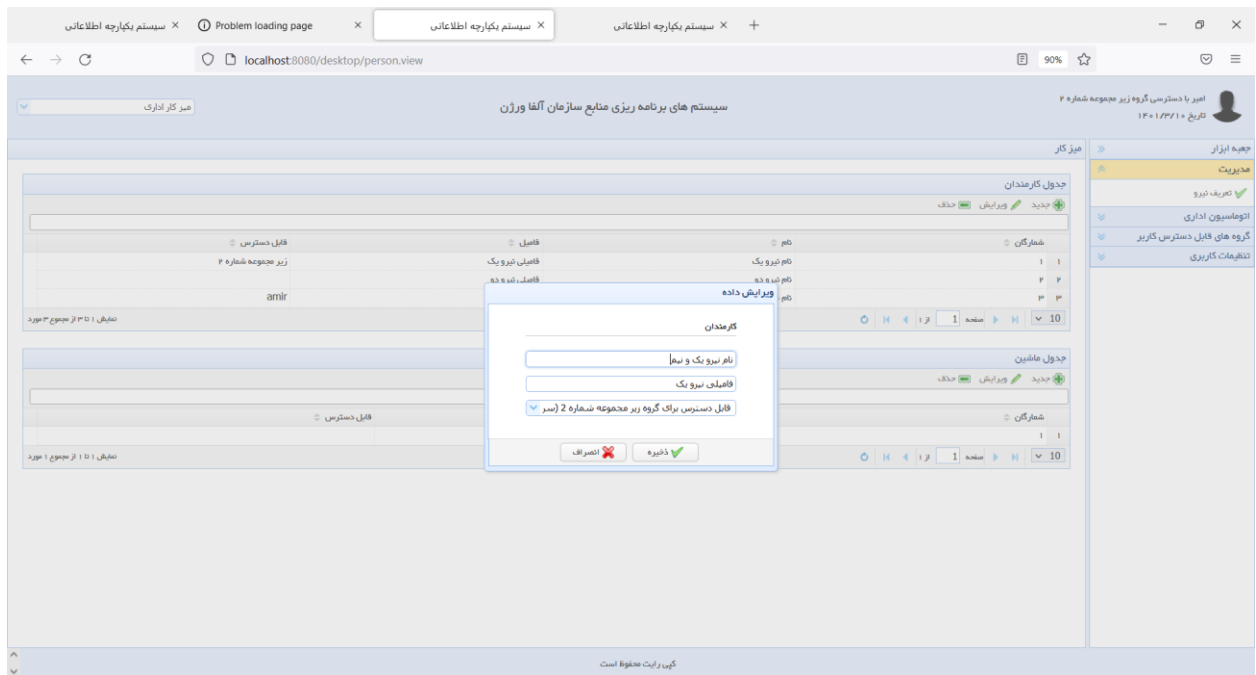
If we change the organizational group now, we'll only see two rows in the employees' table.

The screenshot shows a web application interface with a header bar containing the title "سیستم های برنامه ریزی منابع سازمان آلفا ورژن" and a user profile "امیر با دسترسی گروه زیر مجموعه شماره ۱". The main content area features two tables: "جدول کارمندان" (Employees) and "جدول ماشین" (Machines). The "جدول کارمندان" table has columns for "شماره گن" (ID), "نام" (Name), "قبایل" (Clan), and "قبایل خردو سه" (Clan Xrdw S). It displays two rows: one for "amir" and another for "amir". The "جدول ماشین" table has columns for "شماره گن" (ID) and "مدل" (Model). It displays one row with the value "۱". The sidebar on the right contains a "مدیریت" (Management) section with a "تعریف گروه" (Define Group) button and a list of groups: "اتوماسیون اداری", "گروه های قابل دسترسی کاربر", and "تنظیمات کاربری".

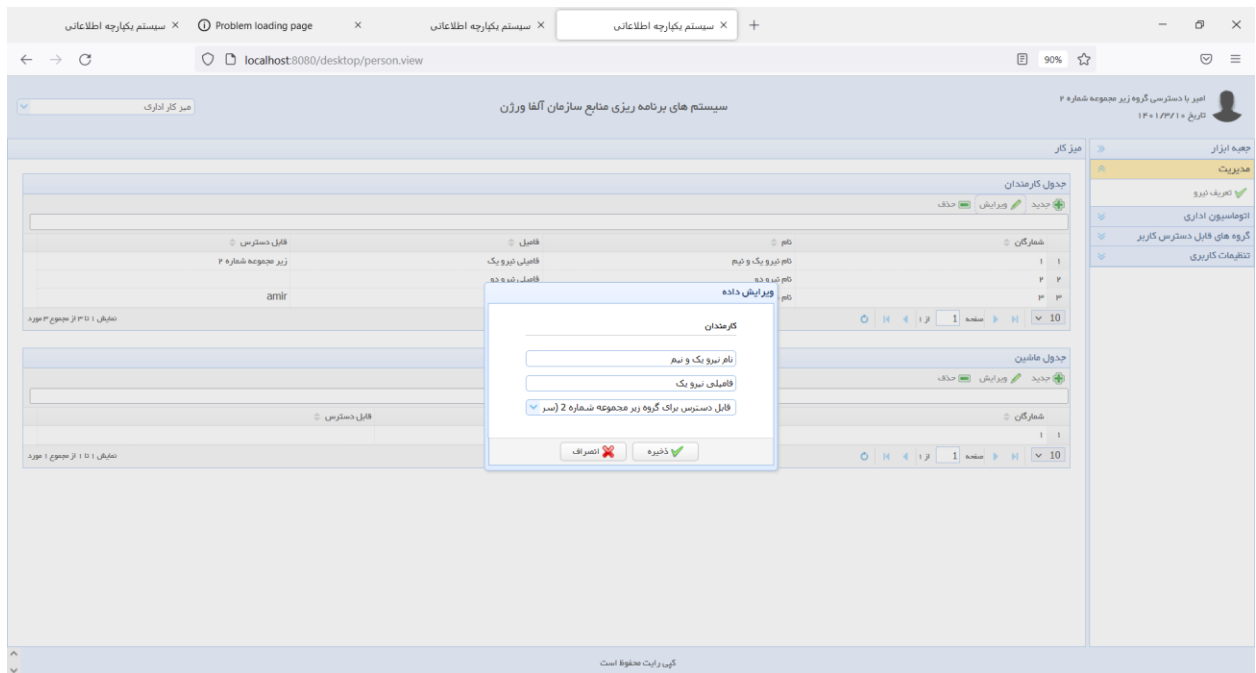
The reason is that the first record was set by Amir as accessible only to "Subgroup No. 2," the second record is accessible to the entire organization, and the third record is visible only to Amir himself. To search records, use the text box above each table. Note that you can also see pagination options at the bottom of each table. Keep in mind that "Subgroup No. 1" has no access to the "name" field, so employee names won't be visible for that group.

The system automatically manages concurrency risks. In the example below, two users attempt to edit simultaneously:

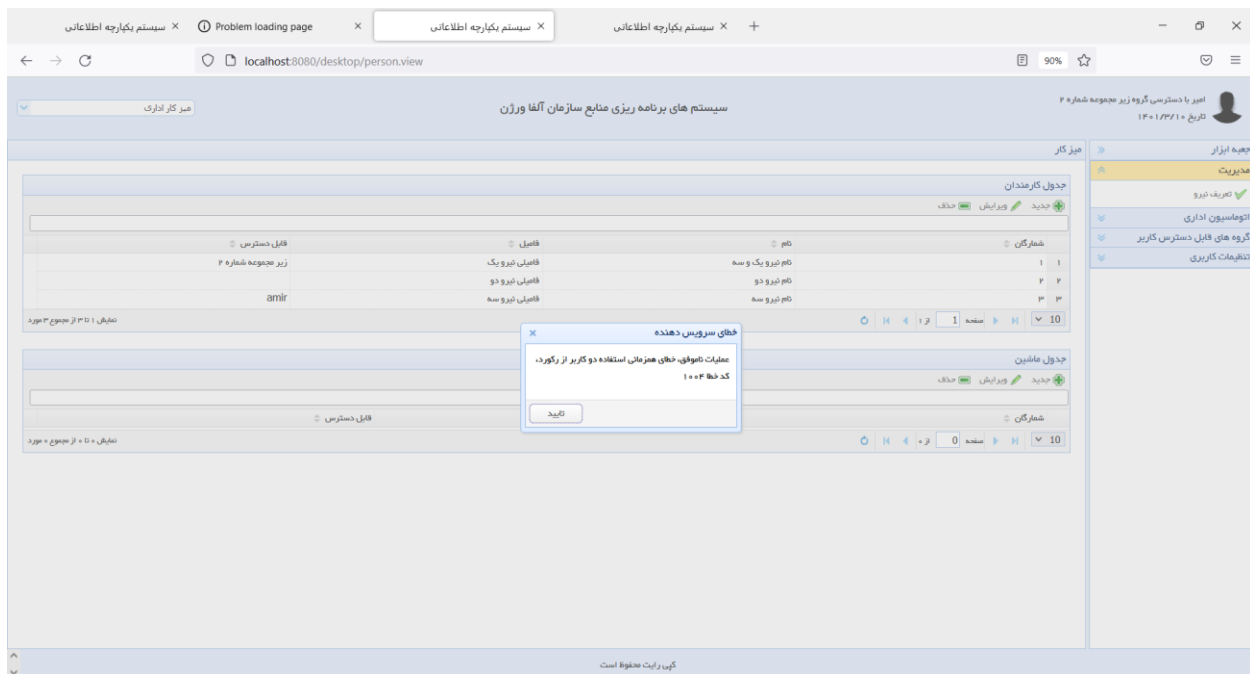
- User1:



- User2:



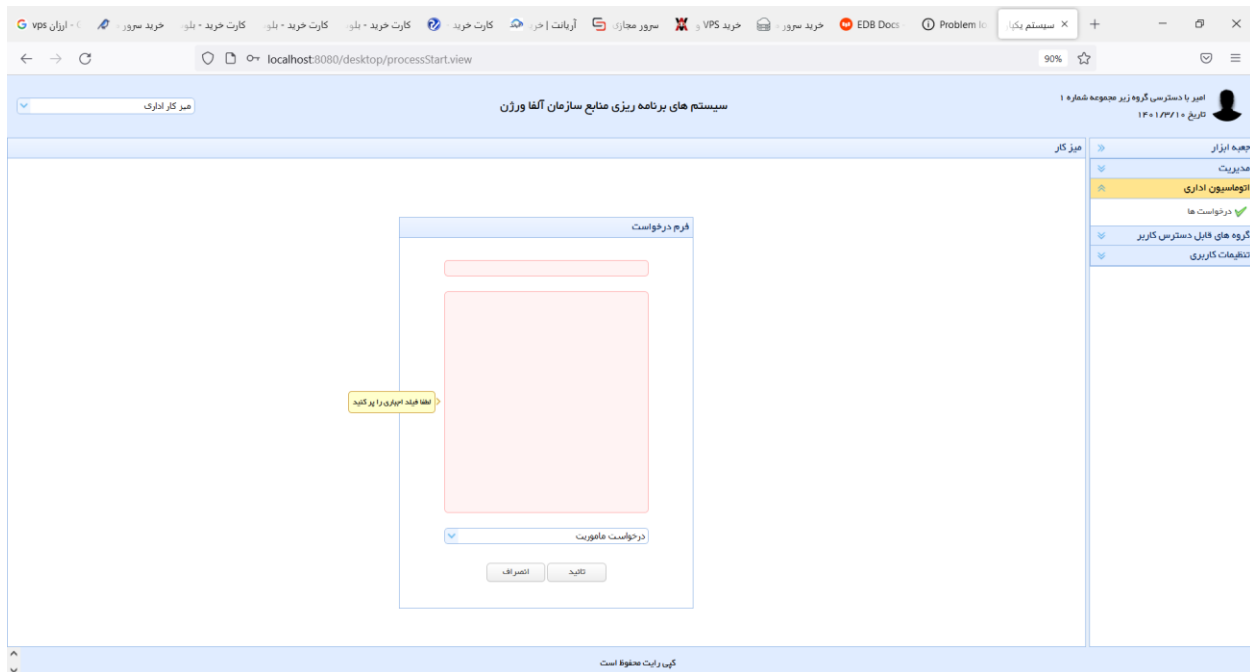
If both users edit simultaneously, the user who saves later will receive an error.



Notably, the developer has no involvement in security, paging, concurrency management, access levels, search, etc.

Process Engine Usage Training

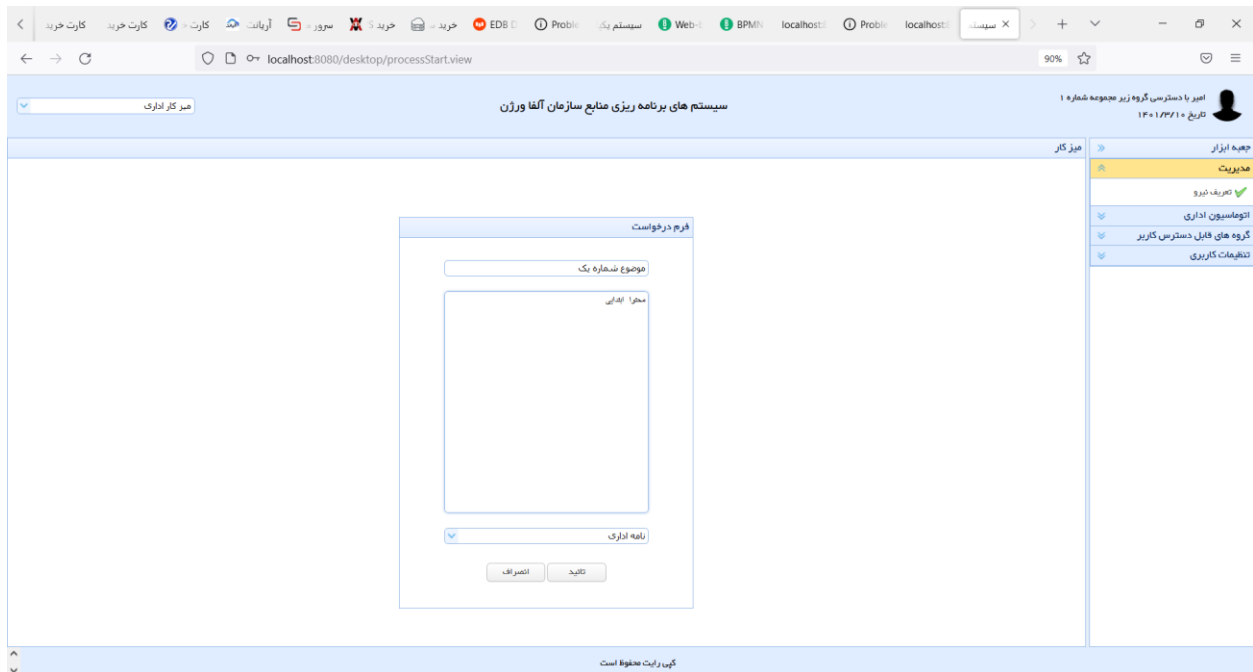
To explore the process management system, first go to the "Administrative Automation" menu. Defined processes based on BPMN2 will be displayed in the process list.



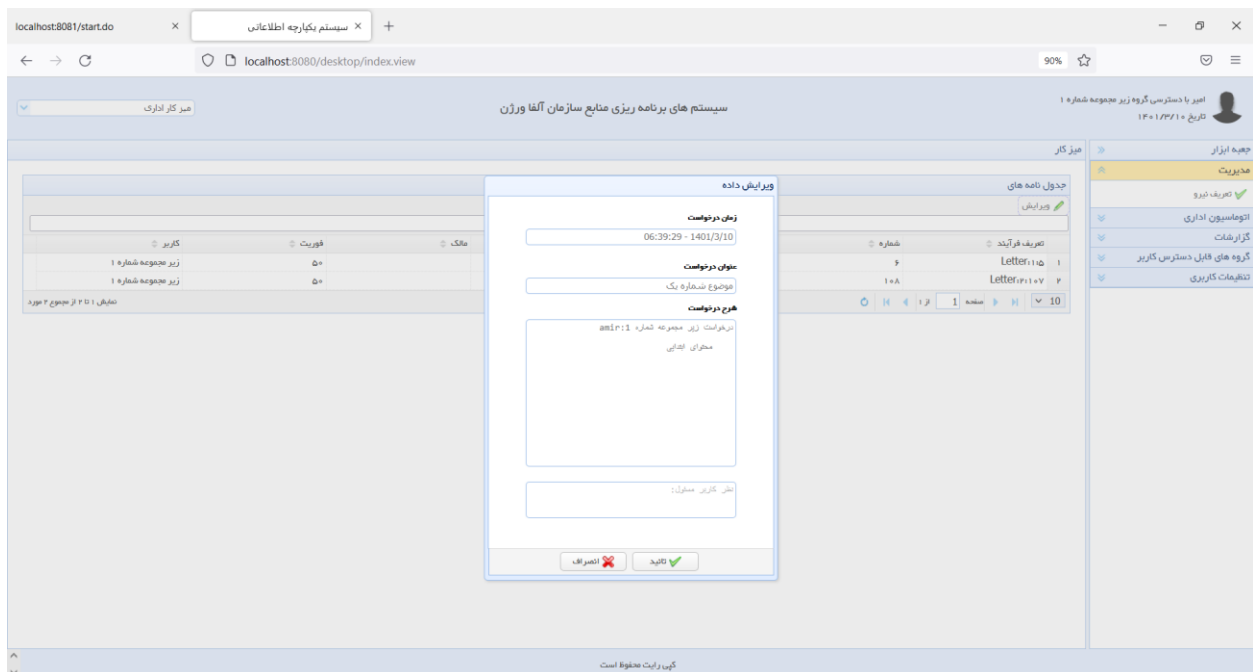
First, select the subject and text for the process, then press the "Confirm" button. The assumed process is called "Leave," and it circulates the document content through the inboxes of "Subgroup No. 1," then "Subgroup No. 2," and finally sends an SMS via a web service provided by the developer. The bpmn.io tool is used for process modeling.



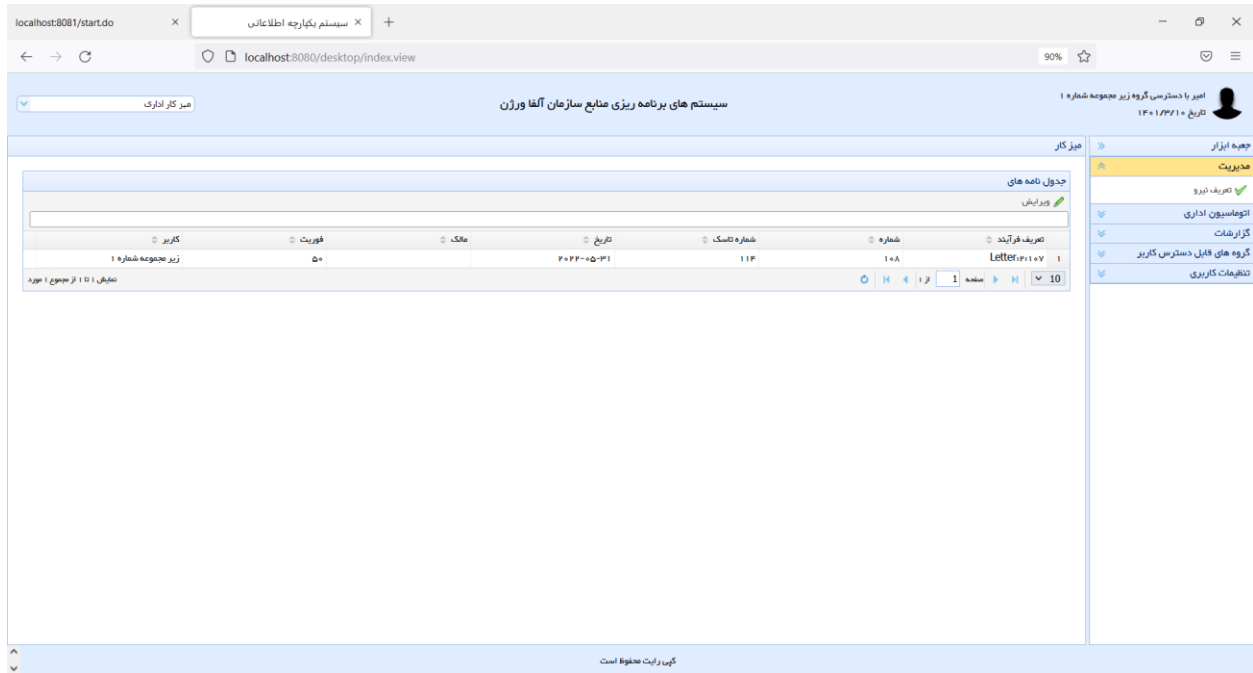
Amir initiates the process. Since Amir is in "Subgroup No. 1," he'll first see the process in his own inbox. As shown below, the process appears in the "Subgroup No. 1" inbox. Amir can now enter his opinion and issue the command to continue the process.



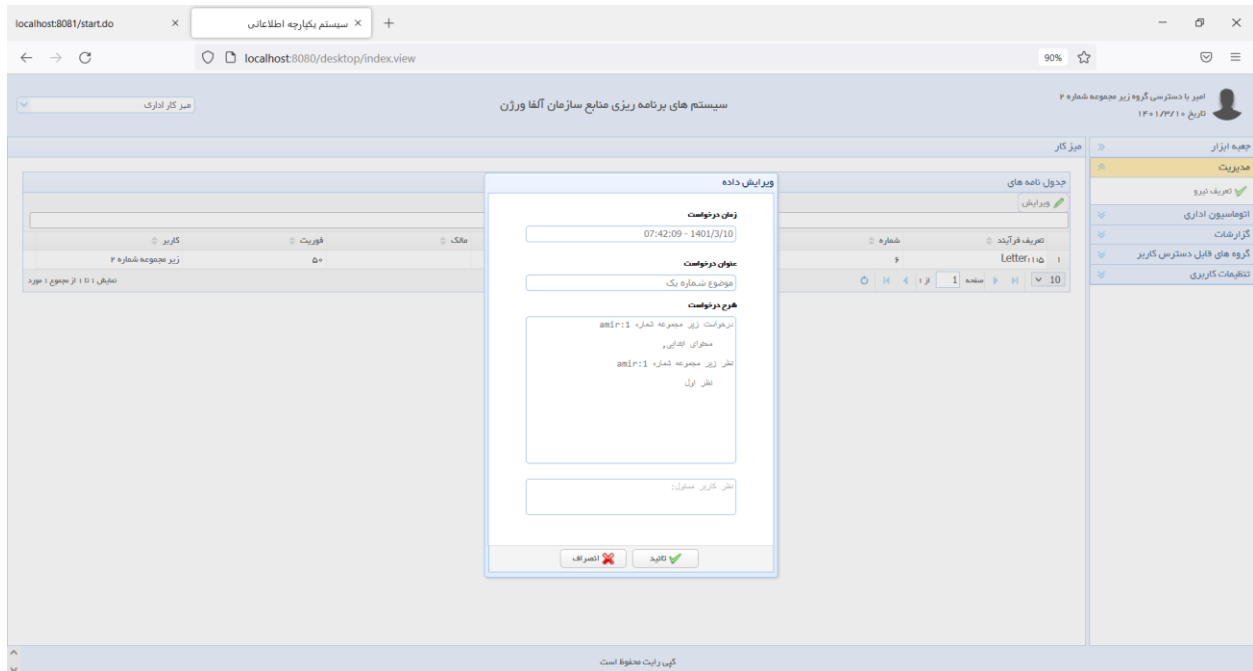
Now, as a member of "Subgroup No. 1," Amir can attach his opinion and confirm the process continuation by pressing "Confirm."



As you can see, the process has left the "Subgroup No. 1" inbox. Now, change Amir's group.



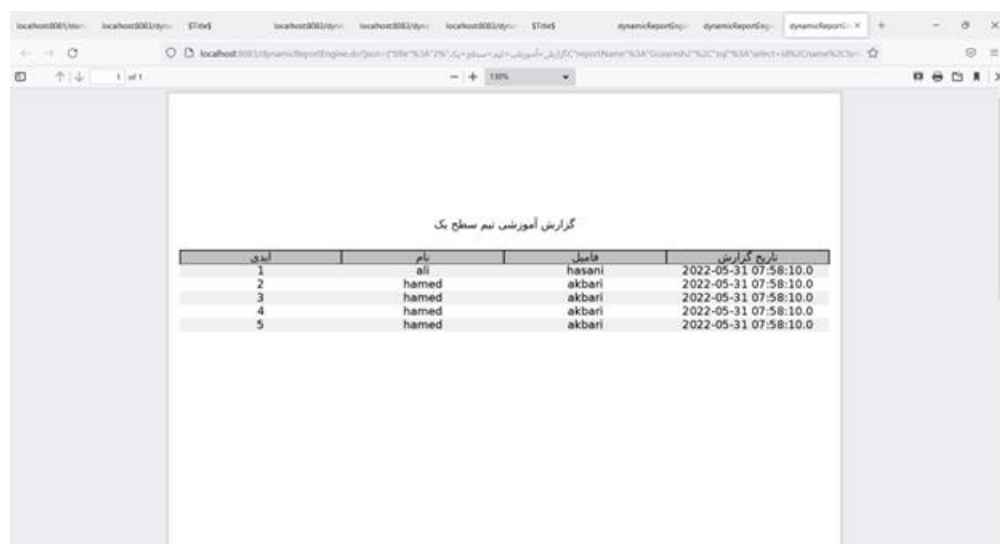
As shown, the previous opinion has been attached, and now the process needs to be moved forward again.



Report Generator Training

Now, let's briefly explain the report generation system. In the "Reports" menu, there are three types of reports, created in three ways:

1. **Type 1:** A report designed in Jasper Studio and added to the system, with SQL commands embedded in the report.
2. **Type 2:** A report designed in Jasper Studio and added to the system, without SQL commands; an entity must be sent to the report.
3. **Type 3:** Reports automatically generated by the system based on an SQL command.



ردیف	نام	فامیل	تاریخ گزارش
1	ali	hasani	2022-05-31 07:58:10.0
2	hamed	akbari	2022-05-31 07:58:10.0
3	hamed	akbari	2022-05-31 07:58:10.0
4	hamed	akbari	2022-05-31 07:58:10.0
5	hamed	akbari	2022-05-31 07:58:10.0

Software Programming Training

The infrastructure's authentication system is based on Jakarta-EE Security and is resistant to attacks like XSS and SQL Injection without developer intervention.

Access Repository Methods List

```

public interface AccessRepository {
    boolean allowSelect(Class aClass);

    boolean allowDelete(Class aClass);

    boolean allowInsert(Class aClass);

    boolean allowUpdate(Class aClass);

    Object preparedFieldOnPermission(Object object) throws NoSuchFieldException, IllegalAccessException;

    List preparedFieldOnPermission(List list) throws NoSuchFieldException, IllegalAccessException;

    void log(Object object, String operation) throws JsonProcessingException, IllegalAccessException;
}

```

These methods are used in the infrastructure's lower layers. The allow methods determine whether operations can be performed on an entity based on the user's organizational group. The prepared methods adjust field access based on the user's group. The log method is for logging. These methods are not typically used by regular developers but are for infrastructure development.

Crud Repository Methods

```

public interface CRUDRepository {

    Object findOneByIdOnPermission(Object object) throws IllegalAccessException, NoSuchFieldException, PermissionException;

    Object findOneByIdOnPermission(Object object, String where, Map map) throws IllegalAccessException, NoSuchFieldException, PermissionException;

    Map findAllOnPermission(Class aClass, String orderBy, PageRequest pageRequest) throws PermissionException, NoSuchFieldException, IllegalAccessException;

    List findAllOnPermission(Class aClass, String orderBy) throws NoSuchFieldException, IllegalAccessException, PermissionException;

    Map findAllOnPermission(Class aClass, String orderBy, String where, Map map, PageRequest pageRequest) throws PermissionException, NoSuchFieldException, IllegalAccessException;

    List findAllOnPermission(Class aClass, String orderBy, String where, Map map) throws NoSuchFieldException, IllegalAccessException, PermissionException;

    Map findAllChildOnPermission(Object object, String childAttributeName, PageRequest pageRequest) throws PermissionException, NoSuchFieldException, IllegalAccessException;

    List findAllChildOnPermission(Object object, String childAttributeName) throws NoSuchFieldException, IllegalAccessException, PermissionException;

    Map findAllChildOnPermission(Object object, String childAttributeName, String where, Map map, PageRequest pageRequest) throws NoSuchFieldException, PermissionException, IllegalAccessException;

    List findAllChildOnPermission(Object object, String childAttributeName, String where, Map map) throws NoSuchFieldException, IllegalAccessException, PermissionException;

    Map searchOnPermission(Class aClass, String orderBy, PageRequest pageRequest) throws PermissionException, NoSuchFieldException, IllegalAccessException;

    Map searchOnPermission(Class aClass, String where, Map map, String orderBy, PageRequest pageRequest) throws PermissionException, NoSuchFieldException, IllegalAccessException;

    Map searchChildOnPermission(Object object, String childAttributeName, PageRequest pageRequest) throws PermissionException, IllegalAccessException, NoSuchFieldException;

    Map searchChildOnPermission(Object object, String childAttributeName, String where, Map map, PageRequest pageRequest) throws PermissionException, NoSuchFieldException, IllegalAccessException;

    Object saveOnPermission(Object object) throws PermissionException, NoSuchFieldException, IllegalAccessException;

    Object deleteOnPermission(Object object) throws PermissionException, IllegalAccessException, NoSuchFieldException, RecordVersionException;

    Object updateOnPermission(Object object) throws PermissionException, IllegalAccessException, NoSuchFieldException, RecordVersionException;
}

```

These methods automatically handle paging, field-level access control, operations, record management, logging, and concurrency resolution. Business developers frequently use these methods. For example, the `findOneByIdOnPermission` method retrieves an object by its primary key using Hibernate, applies field-level access controls, and throws an error if the user lacks viewing permission. The `Map` input specifies query parameters, `orderBy` in `findAll` sets the initial display order, `findChild` uses `ChildAttributeName` to specify the relationship field, and `pageRequest` handles result pagination.

Now, let's implement a simple scenario. First, the developer must define entities. In this example, we use "Person" and "Car" entities.

```
@Entity(name = "person")
@Table(name = "PERSON")
@EntityListeners(Listener.class)
public class Person implements Serializable {
    @Id
    @SequenceGenerator(sequenceName = "PERSON_SEQ", name = "personSeq", allocationSize = 1)
    @GeneratedValue(strategy = GenerationType.AUTO, generator = "personSeq")
    @Column(name = "PERSON_ID", columnDefinition = "NUMBER")
    private Long personId;
    @Column(columnDefinition = "NVARCHAR2(50)")
    private String name;
    @Column(columnDefinition = "NVARCHAR2(50)")
    private String family;

    @ManyToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinTable(name = "VASET", joinColumns = {@JoinColumn(name = "PERSON_ID")}, inverseJoinColumns = {@JoinColumn(name = "CAR_ID")})
    @JsonIgnore
    @OrderBy
    private List<Car> cars;
    @Column(name = "RECORD_VERSION")

    @Scope("prototype")
    @Table(name = "CAR")
    @Entity(name = "car")
    @EntityListeners(Listener.class)
    public class Car implements Serializable {
        @Id
        @SequenceGenerator(name = "carSeq", sequenceName = "CAR_SEQ")
        @GeneratedValue(strategy = GenerationType.AUTO, generator = "carSeq")
        @Column(name = "CAR_ID", columnDefinition = "NUMBER")

        private Long carId;
        @Column(columnDefinition = "NVARCHAR2(10)")
        private String model;
        @Column(name = "RECORD_VERSION")
        @Version
        private Long recordVersion;
        @Column(name = "RECORD_OWNER", columnDefinition = "NVARCHAR2(50)")
        private String recordOwner;
    }
}
```

Next, implement the business logic in the service layer, where the utility of Crud Repository methods becomes clear.

```
@Autowired
private CRUDRepository crudRepository;

public Object save(Person person) {
    try {
        return crudRepository.saveOnPermission(person);
    } catch (Exception e) {
        return ExceptionWrapper.getErrorMessage(e);
    }
}
```

```

public Object getRecordOwnerList() throws JSONException {
    return WebContext.getCurrentRecordOwnerList();
}

public Object delete(Person person) {
    try {
        person = (Person) crudRepository.findOneByIdOnPermission(person);
        if (person.getCars().size() > 0) {
            throw new DependencyException("جدول خودرو بررسی شود");
        }
        crudRepository.deleteOnPermission(person);
        return person;
    } catch (Exception e) {
        return ExceptionWrapper.getErrorMessage(e);
    }
}

public Object update(Person person) {
    try {
        return crudRepository.updateOnPermission(person);
    } catch (Exception e) {
        return ExceptionWrapper.getErrorMessage(e);
    }
}

public Object findOne(Person person) {
    try {
        return crudRepository.findOneByIdOnPermission(person, ENTITY_WHERE_RECORD_OWNER, getUserNameAndGroup());
    } catch (Exception e) {
        return ExceptionWrapper.getErrorMessage(e);
    }
}

public Object read(PageRequest pageRequest) {
    try {
        return crudRepository.findAllOnPermission(Person.class, orderBy: "personId", ENTITY_WHERE_RECORD_OWNER, getUserNameAndGroup(), pageRequest);
    } catch (Exception e) {
        return ExceptionWrapper.getErrorMessage(e);
    }
}

public Object search(PageRequest pageRequest) {
    try {
        return crudRepository.searchOnPermission(Person.class, ENTITY_WHERE_RECORD_OWNER, getUserNameAndGroup(), orderBy: "personId", pageRequest);
    } catch (Exception e) {
        return ExceptionWrapper.getErrorMessage(e);
    }
}

```

As you can see, these methods automatically handle critical production aspects without developer intervention. For instance, the Search method incorporates security, logging, paging, and concurrency resolution. Now, the developer creates REST services using the service layer methods. If a WebAPI architecture is used, a JavaScript library is provided for frontend generation.

```

<table ID="primaryDataTable" title="جدول کارمندان" class="datagrid" style="..."
    url="/desktop/person/read.do"
    toolbar="#primaryToolbar"
    pagination="true"
    rownumbers="true"
    fitColumns="true"
    singleSelect="true"
    remoteSort="false">
    <thead>
    <tr>
    <th data-options="sortable:true" field="personId" width="50">شمارگان</th>
    <th data-options="sortable:true" field="name" width="50">نام</th>
    <th data-options="sortable:true" field="family" width="50">نامیلى</th>
    <th data-options="sortable:true" field="recordOwner" width="50">فنايل دستريم</th>
    </tr>
    </thead>
</table>
<div ID="primaryToolbar">
    <a href="javascript:void(0)" class="linkbutton" iconCls="icon-add" plain="true"
    onclick="newRecord('primaryDialog','primaryForm','/desktop/person/save.do')">جديد</a>
    <a href="javascript:void(0)" class="linkbutton" iconCls="icon-edit" plain="true"
    onclick="editRecord('primaryDataTable','primaryForm','primaryDialog','/desktop/person/update.do','/desktop/person/findOne.do','personId')">ويراين</a>
    <a href="javascript:void(0)" class="linkbutton" iconCls="icon-remove" plain="true"
    onclick="destroyRecord('primaryDataTable','/desktop/person/delete.do','personId','slaveDataTable')">حذف</a>

    <input ID="primarySearchBox" type="text" style="..."
    onkeyup="searchRecord('primaryDataTable','primarySearchBox','/desktop/person/search.do')">

</div>

<div ID="primaryDialog" class="dialog" data-options="closable:false,modal:true" style="..."
    closed="true" buttons="#primaryDialogButtons">
    <form ID="primaryForm" method="post" novalidate style="...">
    <div style="...">كارمندان</div>
    <div style="...">
    <input name="name" class="textbox" required="true" data-options="prompt:'نام'
    style="...">
    </div>
    <div style="...">
    <input name="family" class="textbox" data-options="prompt:'نامیلى'" style="...">
    </div>
    <div style="...">
    <input class="combobox" name="recordOwner" style="..." data-options="
    prompt:'سطح محرماتكى اطلاعات',
    url:'/desktop/person/getRecordOwnerList.do',
    method:'get',
    valueField:'recordOwnerValue',
    textField:'recordOwnerLabel',
    panelHeight:'auto',
    editable:false,
    required:true">
    </div>

    <div style="...">
    <input type="hidden" name="recordVersion">
    </div>
    </form>
</div>

```

The UI code for the employees' table uses this JavaScript library. Note that error handling on the server side is managed by the ExceptionWrapper class, and the developer only needs to specify field names in the UI.

Each panel has a template structure, configured as follows:

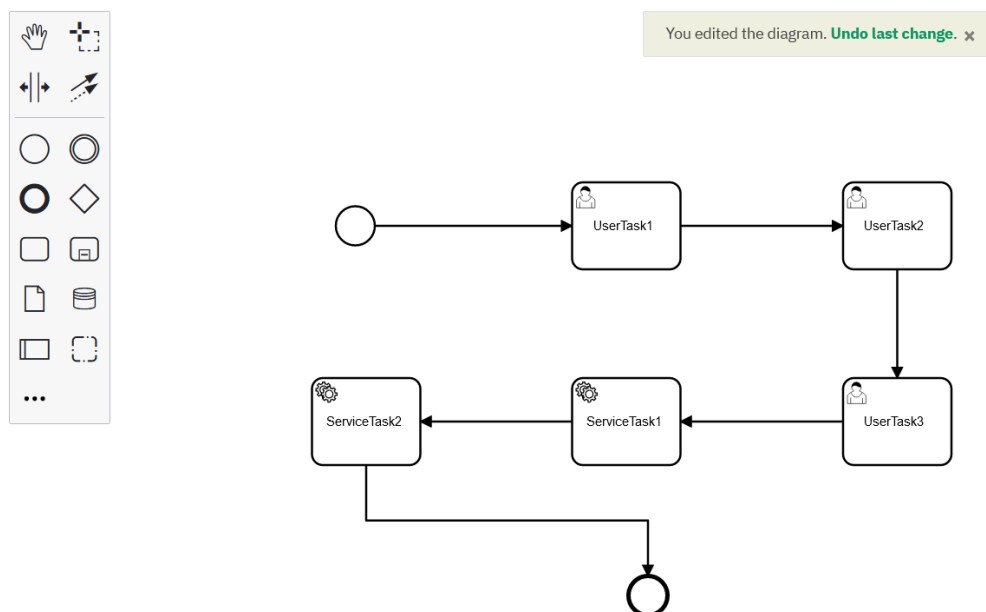
```

<definition name="desktop/index.view" template="/desktop/index.jsp">
  <put-attribute name="up" value="/desktop/up.jsp"/>
  <put-attribute name="right" value="/desktop/right.jsp"/>
  <put-attribute name="content" value="/desktop/processPortal.jsp"/>
  <put-attribute name="down" value="/desktop/down.jsp"/>
</definition>
<definition name="desktop/processStart.view" extends="desktop/index.view">
  <put-attribute name="content" value="/desktop/processStart.jsp"/>
</definition>
<definition name="desktop/processPortal.view" extends="desktop/index.view">
  <put-attribute name="content" value="/desktop/processPortal.jsp"/>
</definition>
<definition name="desktop/person.view" extends="desktop/index.view">
  <put-attribute name="content" value="/desktop/person.jsp"/>
</definition>
<definition name="desktop/property.view" extends="desktop/index.view">
  <put-attribute name="content" value="/desktop/property.jsp"/>
</definition>

```

Now, let's examine the process engine. In this infrastructure, you simply design the process using BPMN2 and place it in the process engine's bpmn folder.

The open-source BPMN.IO tool is suitable for this. Place its output in the process engine's bpmn folder.

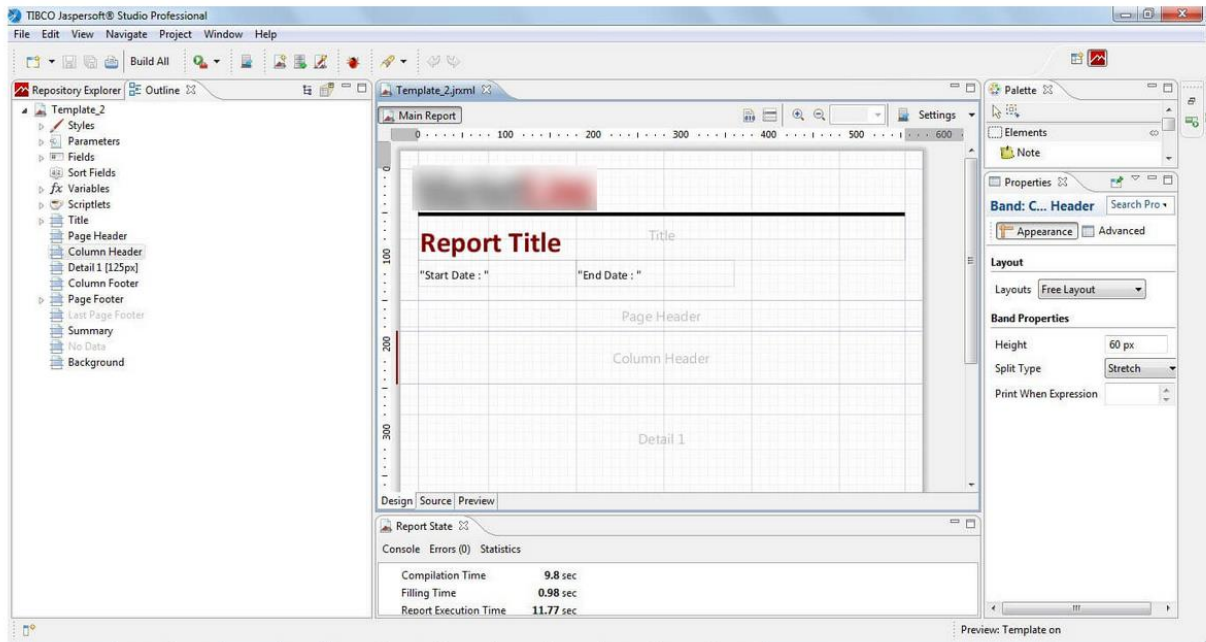


You can use the BPMN Modeler for process modeling. Note that processes must be designed based on an entity called "ProcessDocument." The process engine's remote methods include:

```
public interface BPMSRemoteEngine extends Remote {
    String startProcessDefinitionInstanceById(String processDefinitionId)throws Exception;
    String startProcessDefinitionInstanceById(String processDefinitionId, Map map)throws Exception;
    @Deprecated
    void signalProcessInstance(String processInstanceId)throws Exception;
    @Deprecated
    void signalProcessInstance(String processInstanceId, Map map)throws Exception;
    void completeProcessInstanceTask(String taskId)throws Exception;
    void completeProcessInstanceTask(String taskId, Map map)throws Exception;
    void moveProcessInstance(String processInstanceId, String bpmnUnitId)throws Exception;
    List<BPMSTask> getAllTask()throws Exception;
    List<BPMSTask> getTaskByAssignee(String userOrGroup)throws Exception;
    List<BPMSTask> getTaskById(String processInstanceId)throws Exception;
    List<BPMSTask> getTaskByProcessDefinitionId(String processDefinitionId)throws Exception;
    Map getVariables(String processInstanceId)throws Exception;
    Object getVariable(String processInstanceId, String variableName)throws Exception;
    void setVariables(String processInstanceId, Map map)throws Exception;
    void setVariable(String processInstanceId, String variableName, Object object)throws Exception;
    void suspendProcessDefinition(String processDefinitionId)throws Exception;
    void activeProcessDefinition(String processDefinitionId)throws Exception;
    String addProcessDefinitionFromServerStorage(String bpmnFile, String pngFile)throws Exception;
    String addProcessDefinitionFromSource(String bpmnFile, String source)throws Exception;
    List<String> getActiveProcessDefinition()throws Exception;
    List<String> getSuspendProcessDefinition()throws Exception;
}
```

If you need to replace the process engine, no code rewriting is required. The infrastructure uses the above methods for process engine integration.

For report generation, use Jasper Studio and place the report output in the report folder. This is only needed for non-dynamic reports; the system also supports dynamic report creation. The image below shows the Jasper Studio environment.



Now, let's explain report execution with all three methods:

```
public static void main(String[] args) throws UnsupportedEncodingException, JsonProcessingException {
    Report report = new Report();
    report.setReportType(Report.PDF);
    report.setReportName("Blank_A2");
    System.out.println("http://localhost:8083/reportEngine.do?json="+report.toJsonURLString());
    ///////////////////////////////////////////////////
    EntityReport personEntityReport = new EntityReport();
    personEntityReport.setEntities(Arrays.asList(new Person( id: 1L, name: "هادي", family: "Bahador"), new Person( id: 2L, name: "Maraj", family: "bAHADOR")));
    HashMap hashMap = new HashMap();
    hashMap.put("x", new Date().toString());
    hashMap.put("y", "J20S");
    personEntityReport.setReportName("Blank_A4");
    personEntityReport.setReportType(EntityReport.PDF);
    personEntityReport.setParameters(hashMap);
    System.out.println("http://localhost:8083/entityReportEngine.do?json="+personEntityReport.toJsonURLString());
    ///////////////////////////////////////////////////
    DynamicReport dynamicReport = new DynamicReport()
        .addColumn( name: "id", title: "ايدى")
        .addColumn( name: "name", title: "نام")
        .addColumn( name: "family", title: "فاميل")
        .addColumn( name: "rd", title: "تاريخ گزارش")
        .setReportName("Gozaresh2")
        .setSql("select id,name,family,sysdate rd from person")
        .setTitle("گزارش آموزنى تبم سطح يك")
        .setReportType(DynamicReport.PDF);
    System.out.println("http://localhost:8083/dynamicReportEngine.do?json="+dynamicReport.toJsonURLString());
}
}
```

In the first and second methods, the report is pre-designed in Jasper Studio. In the first method, the SQL command is in the report; in the second, the report lacks SQL, and a list of entities is created via ORM tools and sent to the report for display. Finally, a URL is generated and provided to the user (in this example, URLs are printed to the console). In the third method, the report isn't pre-designed; you send an SQL command to the report engine, which generates the report based on the defined inputs and provides a URL. The output of these commands is shown below.


```
Process finished with exit code 0
```

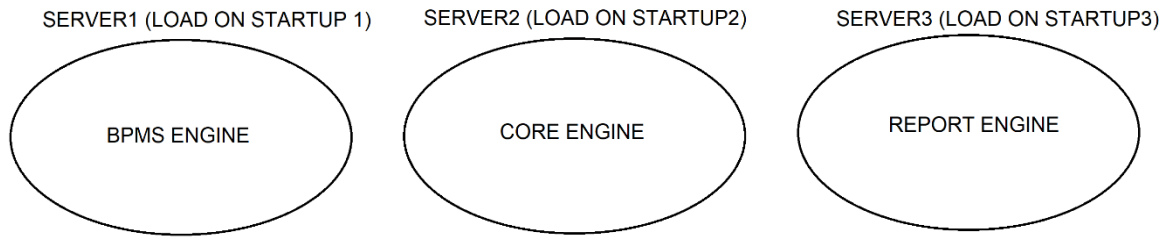
Simple Deployment Training

- Core Infrastructure
- Process Engine
- Report Generator

Tools Used in This Project

- Hibernate ORM Framework
- Spring Framework
- Camunda BPMS Engine
- JasperReport Libraries

Deployment Diagram



NGINX can also be used, and each of the above servers can be scaled to multiple instances.

For any questions or support, contact us at info@j2os.org.